

9. Vue项目实战（三） 物料与图表渲染器

课程资料



byelide.zip

2.48MB



<https://github.com/encode-studio-fe/encode-byelide>

课程源码，重点看这几个分支：

- features/echarts
- features/massive-data-source
- features/multi-chart-renerer

v1.2

课程目标



- 初级：
 - 掌握可视化物料的开发思路
 - 了解 echarts 双引擎(svg、canvas)，并了解 d3、zrender 等
 - 了解离线应用本地存储思路
- 中级：
 - 掌握 echarts 开发配置
 - 掌握图表 schema 设计
 - 掌握 svg 与 canvas 自定义图表实现思路
- 高级：
 - 掌握自定义图表技术选型与设计
 - 以 echarts 为例，了解可视化库原理
 - 熟练掌握 d3、zrender 或者其他三方库实现丰富可视化场景

注意：因为是实战课，除了看懂项目架构与基础开发以外，还要着重培养项目相关描述，在面试过程中这点非常重要！

本节课我们主要从业务层面出发，让大家深度参与一个项目从需求阶段到落地的整个实施过程。

💡 特别是之前很多同学没有参与过大型项目的，主要写后台相关 CRUD 的同学，一定一定要完全掌握本次实战课程的内容！这会直接决定你的编程思维与薪资涨幅！

课程大纲

1. 可视化物料
2. 图表渲染器引擎开发
3. 定制化图表开发（基于 d3/zrender）



加载失败，请重试

课程内容

💡 注意，以下部分代码需结合源码一起查看，此处只展示本章节核心源码。

可视化物料

为项目增加可视化物料，非常简单，我们前面已经将物料设计为插件，在这里我们只需要定义图表 block 物料插件，然后注册即可。

定义可视化物料插件

注册代码实现

```
1  {  
2    type: 'chart',  
3    material: ChartBlock  
4  },
```

```

1  // src/setup.ts
2
3  const baseBlocks = [
4    {
5      type: 'quote',
6      material: QuoteBlock
7    },
8    {
9      type: 'heroTitle',
10     material: HeroTitleBlock
11   },
12   {
13     type: 'view',
14     material: ViewBlock
15   },
16   {
17     type: 'chart',
18     material: ChartBlock
19   },
20   {
21     type: 'image',
22     material: ImageBlock
23   }
24 ]
25 // 因为我们后面会考虑插件市场，所以我们需要一个类来管理所有的 block
26 // 只有你安装了对应的外部插件，你才能在页面中使用

```

定义可视化物料资源

chartBlock 中定义基础内容

```

1  <script setup lang="ts">
2    import ChartRenderer from '@components/ChartRenderer/ChartRenderer.vue'
3  </script>
4
5  <template>
6    <div class="chart">
7      <ChartRenderer />
8    </div>
9  </template>
10
11 <style scoped>

```

```
12  .chart {  
13    width: 100%;  
14  }  
15  </style>  
16
```

我们的核心，就是需要封装这里所看到的 `ChartRenderer`。

图表渲染器我们需要支持如下场景：

1. 基础图表，直接使用 echarts 进行封装，这里我们简单挑一个 echarts 图表做示例
2. 基于 zrender 实现自定义可视化内容
3. 基于 d3 实现地图可视化

 怎么选，svg vs canvas

1. svg，矢量图，具备 DOM 结构的因此如果有比较复杂的 DOM 动画交互我们可以用它，缺点：因为他是真实 DOM，所以一旦内容过多就会卡顿。
2. canvas，位图，通过绘制完成可视化，性能瓶颈主要在**绘图算法**上，缺点：因为不存在真实 DOM，所以动画交互相对比较麻烦。Blender 去做模型，使用 canvas 绘制渲染并粒子、相机、纹理。

右侧物料配置栏，我们简化为可选图表类型，我们暂时实现以上三种图表类型。

使用 echarts 我们可以直接使用 echarts 进行组件封装，或者使用基于 echarts 封装的 vue 组件进行开发，本节课程我们选用后者方案——`vue-echarts`

图表渲染

可视化组件，最核心的内容就是数据加工处理，通常后端返回数据趋于通用，我们需要考虑数据与图表本身配置的解耦合，因此我们需要在前端编写一层数据处理层。从而将处理后的数据喂给图表渲染引擎进行渲染。

我们先来看一个复杂点的图表是怎样的数据把：

```
1  {  
2    color: ['#80FFA5', '#00DDFF', '#37A2FF', '#FF0087', '#FFBF00'],  
3    title: {  
4      text: '渐变堆叠面积图'4
```

```
5     },
6     tooltip: {
7         trigger: 'axis',
8         axisPointer: {
9             type: 'cross',
10            label: {
11                backgroundColor: '#6a7985'
12            }
13        }
14    },
15    legend: {
16        data: ['Line 1', 'Line 2', 'Line 3', 'Line 4', 'Line 5']
17    },
18    toolbox: {
19        feature: {
20            saveAsImage: {}
21        }
22    },
23    grid: {
24        left: '3%',
25        right: '4%',
26        bottom: '3%',
27        containLabel: true
28    },
29    xAxis: [
30        {
31            type: 'category',
32            boundaryGap: false,
33            data: ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
34        }
35    ],
36    yAxis: [
37        {
38            type: 'value'
39        }
40    ],
41    series: [
42        {
43            name: 'Line 1',
44            type: 'line',
45            stack: 'Total',
46            smooth: true,
47            lineStyle: {
48                width: 0
49            },
50            showSymbol: false,
51            areaStyle: {
```

```

52         opacity: 0.8,
53         color: new graphic.LinearGradient(0, 0, 0, 1, [
54             {
55                 offset: 0,
56                 color: 'rgb(128, 255, 165)'
57             },
58             {
59                 offset: 1,
60                 color: 'rgb(1, 191, 236)'
61             }
62         ])
63     },
64     emphasis: {
65         focus: 'series'
66     },
67     data: [140, 232, 101, 264, 90, 340, 250]
68 },
69 {
70     name: 'Line 2',
71     type: 'line',
72     stack: 'Total',
73     smooth: true,
74     lineStyle: {
75         width: 0
76     },
77     showSymbol: false,
78     areaStyle: {
79         opacity: 0.8,
80         color: new graphic.LinearGradient(0, 0, 0, 1, [
81             {
82                 offset: 0,
83                 color: 'rgb(0, 221, 255)'
84             },
85             {
86                 offset: 1,
87                 color: 'rgb(77, 119, 255)'
88             }
89         ])
90     },
91     emphasis: {
92         focus: 'series'
93     },
94     data: [120, 282, 111, 234, 220, 340, 310]
95 },
96 {
97     name: 'Line 3',
98     type: 'line',

```

```
99     stack: 'Total',
100     smooth: true,
101     lineStyle: {
102         width: 0
103     },
104     showSymbol: false,
105     areaStyle: {
106         opacity: 0.8,
107         color: new graphic.LinearGradient(0, 0, 0, 1, [
108             {
109                 offset: 0,
110                 color: 'rgb(55, 162, 255)'
111             },
112             {
113                 offset: 1,
114                 color: 'rgb(116, 21, 219)'
115             }
116         ])
117     },
118     emphasis: {
119         focus: 'series'
120     },
121     data: [320, 132, 201, 334, 190, 130, 220]
122 },
123 {
124     name: 'Line 4',
125     type: 'line',
126     stack: 'Total',
127     smooth: true,
128     lineStyle: {
129         width: 0
130     },
131     showSymbol: false,
132     areaStyle: {
133         opacity: 0.8,
134         color: new graphic.LinearGradient(0, 0, 0, 1, [
135             {
136                 offset: 0,
137                 color: 'rgb(255, 0, 135)'
138             },
139             {
140                 offset: 1,
141                 color: 'rgb(135, 0, 157)'
142             }
143         ])
144     },
145     emphasis: {
```

```

146         focus: 'series'
147     },
148     data: [220, 402, 231, 134, 190, 230, 120]
149 },
150 {
151     name: 'Line 5',
152     type: 'line',
153     stack: 'Total',
154     smooth: true,
155     lineStyle: {
156         width: 0
157     },
158     showSymbol: false,
159     label: {
160         show: true,
161         position: 'top'
162     },
163     areaStyle: {
164         opacity: 0.8,
165         color: new graphic.LinearGradient(0, 0, 0, 1, [
166             {
167                 offset: 0,
168                 color: 'rgb(255, 191, 0)'
169             },
170             {
171                 offset: 1,
172                 color: 'rgb(224, 62, 76)'
173             }
174         ])
175     },
176     emphasis: {
177         focus: 'series'
178     },
179     data: [220, 302, 181, 234, 210, 290, 150]
180 }
181 ]
182 }

```

以上数据包含了一个图表基本的内容配置，包括：颜色、标题、提示组件、图例、工具箱、布局、坐标、数据。

在封装图表时，我们通常最关心的也就是以上几块内容。

图表渲染器

我们上面提到，图表的核心有色值、标题、提示、图例、布局、坐标、数据。

具体配置数据可见：<https://echarts.apache.org/zh/option.html#title>

我们以一个比较复杂的场景，来为大家演示如何使用 echarts 结合 vue 进行图表开发

安装 echarts、vue-echarts

版本，一定一定要注意！

```
1  {  
2      "echarts": "5.4.3",  
3      "vue-echarts": "6.6.0",  
4  }
```

我们注意到，echarts 的按需加载同样使用插件化机制实现，所以插件机制一定一定要记住啊，上节课刚讲，大家可以回看。

```
1  import { use, graphic } from 'echarts/core'  
2  import { CanvasRenderer } from 'echarts/renderers'  
3  import { LineChart } from 'echarts/charts'  
4  import {  
5      TitleComponent,  
6      TooltipComponent,  
7      LegendComponent,  
8      ToolboxComponent,  
9      GridComponent  
10 } from 'echarts/components'  
11 import VChart, { THEME_KEY } from 'vue-echarts'  
12 import { ref, provide } from 'vue'  
13  
14 use([  
15     CanvasRenderer,  
16     LineChart,  
17     TitleComponent,  
18     TooltipComponent,  
19     LegendComponent,  
20     ToolboxComponent,  
21     GridComponent  
22 ])
```

主题注入

vue-echarts 提供了方便的主题注入方式，用到了 provide 和 inject，这个概念大家一定也要会在项目中使用哦。

```
1 import VChart, { THEME_KEY } from 'vue-echarts'
2 import { ref, provide } from 'vue'
3
4 provide(THEME_KEY, 'light')
```

数据定义

```
1
2 const option = ref({
3   color: ['#80FFA5', '#00DDFF', '#37A2FF', '#FF0087', '#FFBF00'],
4   title: {
5     text: '渐变堆叠面积图'
6   },
7   tooltip: {
8     trigger: 'axis',
9     axisPointer: {
10       type: 'cross',
11       label: {
12         backgroundColor: '#6a7985'
13       }
14     }
15   },
16   legend: {
17     data: ['Line 1', 'Line 2', 'Line 3', 'Line 4', 'Line 5']
18   },
19   toolbox: {
20     feature: {
21       saveAsImage: {}
22     }
23   },
24   grid: {
25     left: '3%',
26     right: '4%',
27     bottom: '3%',
28     containLabel: true
29   },
30   xAxis: [
31     {
32       type: 'category',
```

```
33     boundaryGap: false,
34     data: ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
35   }
36 ],
37 yAxis: [
38   {
39     type: 'value'
40   }
41 ],
42 series: [
43   {
44     name: 'Line 1',
45     type: 'line',
46     stack: 'Total',
47     smooth: true,
48     lineStyle: {
49       width: 0
50     },
51     showSymbol: false,
52     areaStyle: {
53       opacity: 0.8,
54       color: new graphic.LinearGradient(0, 0, 0, 1, [
55         {
56           offset: 0,
57           color: 'rgb(128, 255, 165)'
58         },
59         {
60           offset: 1,
61           color: 'rgb(1, 191, 236)'
62         }
63       ])
64     },
65     emphasis: {
66       focus: 'series'
67     },
68     data: [140, 232, 101, 264, 90, 340, 250]
69   },
70   {
71     name: 'Line 2',
72     type: 'line',
73     stack: 'Total',
74     smooth: true,
75     lineStyle: {
76       width: 0
77     },
78     showSymbol: false,
79     areaStyle: {
```

```
80         opacity: 0.8,
81         color: new graphic.LinearGradient(0, 0, 0, 1, [
82             {
83                 offset: 0,
84                 color: 'rgb(0, 221, 255)'
85             },
86             {
87                 offset: 1,
88                 color: 'rgb(77, 119, 255)'
89             }
90         ])
91     },
92     emphasis: {
93         focus: 'series'
94     },
95     data: [120, 282, 111, 234, 220, 340, 310]
96 },
97 {
98     name: 'Line 3',
99     type: 'line',
100     stack: 'Total',
101     smooth: true,
102     lineStyle: {
103         width: 0
104     },
105     showSymbol: false,
106     areaStyle: {
107         opacity: 0.8,
108         color: new graphic.LinearGradient(0, 0, 0, 1, [
109             {
110                 offset: 0,
111                 color: 'rgb(55, 162, 255)'
112             },
113             {
114                 offset: 1,
115                 color: 'rgb(116, 21, 219)'
116             }
117         ])
118     },
119     emphasis: {
120         focus: 'series'
121     },
122     data: [320, 132, 201, 334, 190, 130, 220]
123 },
124 {
125     name: 'Line 4',
126     type: 'line',
```

```
127     stack: 'Total',
128     smooth: true,
129     lineStyle: {
130         width: 0
131     },
132     showSymbol: false,
133     areaStyle: {
134         opacity: 0.8,
135         color: new graphic.LinearGradient(0, 0, 0, 1, [
136             {
137                 offset: 0,
138                 color: 'rgb(255, 0, 135)'
139             },
140             {
141                 offset: 1,
142                 color: 'rgb(135, 0, 157)'
143             }
144         ])
145     },
146     emphasis: {
147         focus: 'series'
148     },
149     data: [220, 402, 231, 134, 190, 230, 120]
150 },
151 {
152     name: 'Line 5',
153     type: 'line',
154     stack: 'Total',
155     smooth: true,
156     lineStyle: {
157         width: 0
158     },
159     showSymbol: false,
160     label: {
161         show: true,
162         position: 'top'
163     },
164     areaStyle: {
165         opacity: 0.8,
166         color: new graphic.LinearGradient(0, 0, 0, 1, [
167             {
168                 offset: 0,
169                 color: 'rgb(255, 191, 0)'
170             },
171             {
172                 offset: 1,
173                 color: 'rgb(224, 62, 76)'
```

```

174         }
175     })
176 },
177     emphasis: {
178         focus: 'series'
179     },
180     data: [220, 302, 181, 234, 210, 290, 150]
181 }
182 ]
183 })

```

渲染器

渲染器根据当前图表类型决定如何渲染，其实又是策略模式的运用，我们通常针对这样的渲染器，命名为：xxxRenderer，这里我们命名为 ChartRenderer，具体如何渲染，是由各类型图表决定的。

```

1  <template>
2    <component :is="renderer" class="chart-container" :block-info="blockInfo" />
3  </template>
4
5  <script setup lang="ts">
6    import EchartsRenderer from './EchartsRenderer/EchartsRenderer.vue'
7    import CanvasChartRenderer from
8      './CanvasChartRenderer/CanvasChartRenderer.vue'
9    import SVGChartRenderer from './SVGChartRenderer/SVGChartRenderer.vue'
10   import { computed } from 'vue'
11   import type { ChartBlockInfo } from '@types/block'
12
13   const props = defineProps<{
14     blockInfo: ChartBlockInfo
15   }>()
16
17   const renderer = computed(() => {
18     if (!props.blockInfo) return ''
19     switch (props.blockInfo.props.chartType) {
20       case 'echarts': {
21         return EchartsRenderer
22       }
23       case 'canvas': {
24         return CanvasChartRenderer
25       }
26       case 'svg': {
27         return SVGChartRenderer
28       }
29     }
30   })

```

```
27     }
28     default:
29         return ''
30     }
31 })
32 </script>
33
34 <!-- <style scoped></style> -->
35
```

定制化图表开发

定制化图表底层就两个大的方案，分别为：canvas、svg。

两者在选择时一般需要考虑以下几点：

1. 图表元素是否需要获取节点，是的话则选择 svg
2. 绘制的图表对于缩放的处理，如果图渐变可以 svg，这是矢量图的优势
3. 绘制数据量，如果数据量特别大，不用考虑直接选择 canvas



为了给大家更直观感受，我整理了一个绘制 1000 万行数据的表格，性能非常好。

很多同学在面试被问到表格的性能优化，可以这样回答

1. 一般情况下我们选择使用分页
2. 如果必须显示全部数据，则考虑虚拟滚动
3. 巨量数据，必须得上 canvas

基于 zrender 开发可视化物料

zrender 是 echarts 底层框架，使用它我们可以非常方便绘制图表元素、处理事件、性能优化等。

安装 zrender

```
1 "zrender": "5.4.4",
```

核心 api

该框架 api 相对简单，我们需要了解以下几个 api

1. init
2. 形状: zrender.Text、zrender.Rect 等
3. add、remove
4. on、off

一个绘制简单示例

```
1  <template>
2    <div class="canvas" ref="containerRef"></div>
3  </template>
4
5  <script setup lang="ts">
6    import * as zrender from 'zrender'
7    import { onMounted, ref } from 'vue'
8
9    var containerRef = ref<HTMLDivElement | null>(null)
10
11    onMounted(() => {
12      var zr = zrender.init(containerRef.value, { renderer: 'svg' })
13
14      var w = zr.getWidth()
15      var h = zr.getHeight()
16
17      var t1 = new zrender.Text({
18        style: {
19          text: 'zrender',
20          align: 'center',
21          verticalAlign: 'middle',
22          fill: '#0ff',
23          fontSize: 200,
24          fontFamily: 'Lato',
25          fontWeight: 'bolder',
26          // blend: 'lighten',
27          x: w / 2 + 5,
28          y: h / 2
29        }
30      })
31      zr.add(t1)
32
33      var t2 = new zrender.Text({
34        culling: true,
35        style: {
36          text: 'zrender',
```



```

37     fontSize: 200,
38     align: 'center',
39     fill: '#fff',
40     verticalAlign: 'middle',
41     fontFamily: 'Lato',
42     fontWeight: 'bolder',
43     x: w / 2,
44     y: h / 2
45   }
46 })
47 zr.add(t2)
48
49 var lines: zrender.Rect[] = []
50 for (var i = 0; i < 16; ++i) {
51   var line = new zrender.Rect({
52     shape: {
53       x: w * (Math.random() - 0.3),
54       y: h * Math.random(),
55       width: w * (Math.random() + 0.3),
56       height: Math.random() * 8
57     },
58     style: {
59       fill: ['#ff0', '#f0f', '#0ff', '#00f'][Math.floor(Math.random() * 4)],
60       blend: 'lighten',
61       opacity: 0
62     }
63   })
64   zr.add(line)
65   lines.push(line)
66 }
67
68 setInterval(function () {
69   if (Math.random() > 0.2) {
70     t2.attr('style', {
71       x: w / 2 + Math.random() * 50,
72       y: h / 2
73     })
74
75     for (var i = 0; i < lines.length; ++i) {
76       lines[i].attr('shape', {
77         x: w * Math.random(),
78         y: h * Math.random(),
79         width: w * Math.random(),
80         height: Math.random() * 8
81       })
82       lines[i].attr('style', {
83         opacity: 1

```

```

84         })
85     }
86
87     setTimeout(function () {
88         t2.attr('style', {
89             x: w / 2,
90             y: h / 2
91         })
92
93         for (var i = 0; i < lines.length; ++i) {
94             lines[i].attr('style', {
95                 opacity: 0
96             })
97         }
98     }, 100)
99 }
100 }, 500)
101 })
102 </script>
103
104 <style scoped>
105 .canvas {
106     width: 100%;
107     height: 360px;
108     background-color: #333;
109 }
110 </style>
111

```

我们将该图表定义在 `CanvasChartRenderer` 中，根据 `chartType` 策略，当 `chartType === 'canvas'` 时渲染。

基于 svg 开发可视化物料

安装 d3

```

1  "d3": "7.8.5"

```

Vue3 ref 源码的理解

<https://github.com/vuejs/core/blob/b775b71c788499ec7ee58bc2cf4cd04ed388e072/packages/runtime-core/src/rendererTemplateRef.ts#L23>

🏆 重点关注的组件：SegmentedControl，分段器组件

1. Props 设计
2. Event 设计
3. 关于副作用的使用

组件基础封装

和使用 zrender 一样，我们首先需要定义图表组件容器元素

```
1  <template>
2    <div class="svg" ref="containerRef"></div>
3  </template>
4
5  <script setup lang="ts">
6    import * as d3 from 'd3'
7    import { onMounted, ref } from 'vue'
8
9    const containerRef = ref<HTMLDivElement | null>(null)
10  </script>
```

请注意，d3 也是数据驱动框架，所以你需要时刻注意当前数据，从而根据数据变更来决定 svg dom 渲染。

```
1    const width = containerRef.value?.clientWidth || 1024
2    const height = containerRef.value?.clientHeight || 768
3
4    const svg = d3
5      .select(containerRef.value)
6      .append('svg')
7      .attr('width', width)
8      .attr('height', height)
9      .append('g')
10     .attr('transform', 'translate(0,0)')
```

使用 d3 绘制图形需要关注 svg dom 的绘制，时刻关注 data

```
1      svg
2      .selectAll('path')
3      // 这是核心!
4      .data(root.features) // 这里的数据需要关注，他的变更决定了后续 dom 的变更
5      .enter()
6      .append('path')
7      .attr('stroke', '#000')
8      .attr('stroke-width', 1)
9      .style('opacity', 0.8)
```

获取地图数据

绘制地图需要有地理数据，通常地理数据是一个 JSON 格式数据，用于描述地图区块围栏。我们使用远程数据：

```
1      d3.json<d3.ExtendedFeatureCollection>(
2
3      'https://gist.githubusercontent.com/zhulinpinyu/8e18d57b3b18fb74e776/raw/efbb74cfea53decbb1fe7d6bf279fd351c28c4810/china_simplify.json'
4      )
```

绘制地图

绘制地图区块，并为区块绑定事件。

```
1      d3.json<d3.ExtendedFeatureCollection>(
2
3      'https://gist.githubusercontent.com/zhulinpinyu/8e18d57b3b18fb74e776/raw/efbb74cfea53decbb1fe7d6bf279fd351c28c4810/china_simplify.json'
4      ).then((root: any) => {
5
6          const featuresLen = root.features.length
7
8          const colors = d3.scaleLinear(
9              [0, featuresLen * 0.33, featuresLen * 0.66, featuresLen],
10             ['#B58929', '#C61C6F', '#268BD2', '#85992C']
11         )
12         // .domain([0, featuresLen * 0.33, featuresLen * 0.66, featuresLen])
```

```

11     // .range(['#B58929', '#C61C6F', '#268BD2', '#85992C'])
12
13     //添加提示
14     const tooltip = d3.select('body').append('div').attr('class',
15 'tooltip').style('opacity', 0)
16
17     //绘制地图
18     svg
19     .selectAll('path')
20     // 这是核心!
21     .data(root.features)
22     .enter()
23     .append('path')
24     .attr('stroke', '#000')
25     .attr('stroke-width', 1)
26     .style('opacity', 0.8)
27     .attr('fill', function (d, i: number) {
28         return colors(i)
29     })
30     // @ts-ignore
31     .attr('d', path)
32     .on('mouseover', function (ev, d) {
33         const hoveredData = d as d3.ExtendedFeature
34         if (!hoveredData) return
35
36         d3.select(this).style('opacity', 1)
37
38         tooltip.transition().duration(200).style('opacity', 0.9)
39
40         tooltip
41             .html(hoveredData.properties?.name)
42             .style('left', ev.pageX + 'px')
43             .style('top', ev.pageY - 28 + 'px')
44     })
45     .on('mouseout', function () {
46         d3.select(this).style('opacity', 0.8)
47         tooltip.transition().duration(500).style('opacity', 0)
48     })

```

完整代码

```
2     <div class="svg" ref="containerRef"></div>
3 </template>
4
5 <script setup lang="ts">
6 import * as d3 from 'd3'
7 import { onMounted, ref } from 'vue'
8
9 const containerRef = ref<HTMLDivElement | null>(null)
10
11 onMounted(() => {
12     const width = containerRef.value?.clientWidth || 1024
13     const height = containerRef.value?.clientHeight || 768
14
15     const svg = d3
16         .select(containerRef.value)
17         .append('svg')
18         .attr('width', width)
19         .attr('height', height)
20         .append('g')
21         .attr('transform', 'translate(0,0)')
22
23     const projection = d3
24         .geoMercator()
25         .center([107, 38])
26         .scale(500)
27         .translate([width / 2, height / 2])
28
29     const path = d3.geoPath(projection)
30
31     d3.json<d3.ExtendedFeatureCollection>(
32         'https://gist.githubusercontent.com/zhulinpinyu/8e18d57b3b18fb74e776/raw/efbb7
33         4cfea53decblfe7d6bf279fd351c28c4810/china_simplify.json'
34     ).then((root: any) => {
35         const featuresLen = root.features.length
36
37         const colors = d3.scaleLinear(
38             [0, featuresLen * 0.33, featuresLen * 0.66, featuresLen],
39             ['#B58929', '#C61C6F', '#268BD2', '#85992C']
40         )
41         // .domain([0, featuresLen * 0.33, featuresLen * 0.66, featuresLen])
42         // .range(['#B58929', '#C61C6F', '#268BD2', '#85992C'])
43
44         //添加提示
45         const tooltip = d3.select('body').append('div').attr('class',
46             'tooltip').style('opacity', 0)
```

```

46     //绘制地图
47     svg
48         .selectAll('path')
49         // 这是核心!
50         .data(root.features)
51         .enter()
52         .append('path')
53         .attr('stroke', '#000')
54         .attr('stroke-width', 1)
55         .style('opacity', 0.8)
56         .attr('fill', function (d, i: number) {
57             return colors(i)
58         })
59         // @ts-ignore
60         .attr('d', path)
61         .on('mouseover', function (ev, d) {
62             const hoveredData = d as d3.ExtendedFeature
63             if (!hoveredData) return
64
65             d3.select(this).style('opacity', 1)
66
67             tooltip.transition().duration(200).style('opacity', 0.9)
68
69             tooltip
70                 .html(hoveredData.properties?.name)
71                 .style('left', ev.pageX + 'px')
72                 .style('top', ev.pageY - 28 + 'px')
73         })
74         .on('mouseout', function () {
75             d3.select(this).style('opacity', 0.8)
76             tooltip.transition().duration(500).style('opacity', 0)
77         })
78     })
79 })
80 </script>
81
82 <style scoped>
83     .svg {
84         width: 100%;
85         height: 560px;
86     }
87 </style>
88
89 <style lang="css">
90     .tooltip {
91         position: absolute;
92         z-index: 9999;

```

```
93     padding: 0;
94     color: const(--color-primary);
95     font-size: 13px;
96     text-align: left;
97     border-radius: 2px;
98     pointer-events: none;
99 }
100 </style>
101
```

d3 相关示例

https://observablehq.com/@d3/gallery?utm_source=d3js-org&utm_medium=nav&utm_campaign=try-observable

本地持久化

通常，我们在前端可以临时缓存数据，而后通过更新队列将数据同步至后端服务。本地存储通常采用的方案有：

1. localStorage
2. indexedDB
3. webSQL，已废弃，不用了解

通常小数据量可以直接选用 localStorage，但是需要注意的是其数据序列化与反序列化的实现。

对于较大数据量，我们可以选用 indexedDB，当然直接操作会略微有点麻烦，我们可以选择使用 **Dexie** <https://github.com/dexie/Dexie.js>

indexedDB + service worker 就能实现本地离线应用